11

DIP: The Dependency-Inversion Principle



Nevermore
Let the great interests of the State depend
Upon the thousand chances that may sway
A piece of human frailty

—Sir Thomas Noon Talfourd (1795–1854)

DIP: The Dependency-Inversion Principle

> *a. High-level modules should not depend on low-level modules. Both should depend on abstractions.*
> *b. Abstractions should not depend on details. Details should depend on abstractions.*

Over the years, many have questioned why I use the word "inversion" in the name of this principle. It is because more traditional software development methods, such as Structured Analysis and Design, tend to create software structures in which high-level modules depend on low-level modules, and in which policy depends on detail. Indeed one of the goals of these methods is to define the subprogram hierarchy that describes how the high-level modules make calls to the low-level modules. The initial design of the `Copy` program in Figure 7-1 on page 90 is a good example of such a hierarchy. The dependency structure of a well-designed, object-oriented program is "inverted" with respect to the dependency structure that normally results from traditional procedural methods.

Consider the implications of high-level modules that depend on low-level modules. It is the high-level modules that contain the important policy decisions and business models of an application. These modules

contain the identity of the application. Yet, when these modules depend on the lower level modules, changes to the lower level modules can have direct effects on the higher level modules and can force them to change in turn.

This predicament is absurd! It is the high-level, policy-setting modules that ought to be influencing the low-level, detailed modules. The modules that contain the high-level business rules should take precedence over, and be independent of, the modules that contain the implementation details. High-level modules simply should not depend on low-level modules in any way.

Moreover, it is high-level, policy-setting modules that we want to be able to reuse. We are already quite good at reusing low-level modules in the form of subroutine libraries. When high-level modules depend on low-level modules, it becomes very difficult to reuse those high-level modules in different contexts. However, when the high-level modules are independent of the low-level modules, then the high-level modules can be reused quite simply. This principle is at the very heart of framework design.

Layering

According to Booch, "...all well-structured object-oriented architectures have clearly defined layers, with each layer providing some coherent set of services though a well-defined and controlled interface."[1] A naive interpretation of this statement might lead a designer to produce a structure similar to Figure 11-1. In this diagram, the high-level `Policy` layer uses a lower-level `Mechanism` layer, which in turn uses a detailed-level `Utility` layer. While this may look appropriate, it has the insidious characteristic that the `Policy` layer is sensitive to changes all the way down in the `Utility` layer. *Dependency is transitive.* The `Policy` layer depends on something that depends on the `Utility` layer; thus, the `Policy` layer transitively depends on the `Utility` layer. This is very unfortunate.

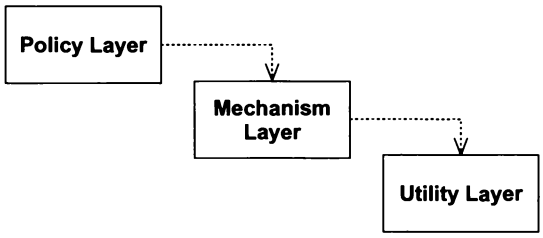


Figure 11-1 Naive layering scheme

Figure 11-2 shows a more appropriate model. Each of the upper-level layers declares an abstract interface for the services that it needs. The lower-level layers are then realized from these abstract interfaces. Each higher-level class uses the next-lowest layer through the abstract interface. Thus, the upper layers do not depend on the lower layers. Instead, the lower layers depend on abstract service interfaces *declared in* the upper layers. Not only is the transitive dependency of `PolicyLayer` on `UtilityLayer` broken, but even the direct dependency of the `PolicyLayer` on `MechanismLayer` is broken.

An Inversion of Ownership

Notice that the inversion here is not just one of dependencies, it is also one of interface ownership. We often think of utility libraries as owning their own interfaces. But when the DIP is applied, we find that the clients tend to own the abstract interfaces and that their servers derive from them.

1. [Booch96], p. 54.

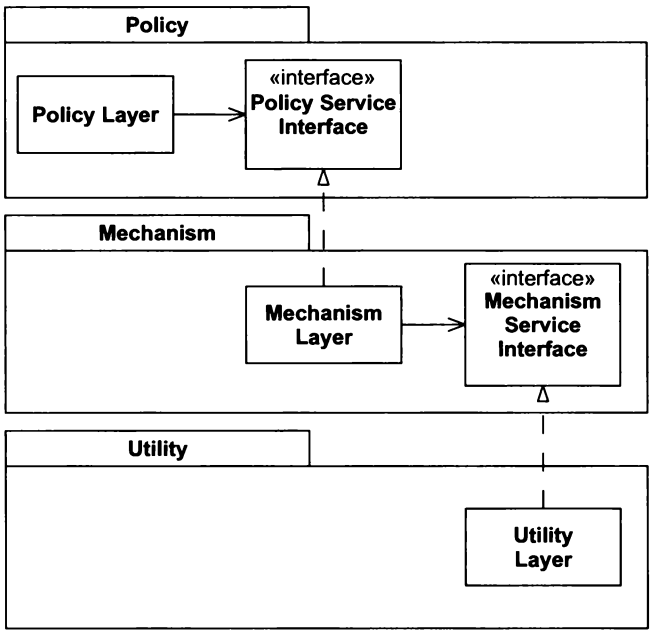


Figure 11-2 Inverted Layers

This is sometimes known as the Hollywood principle: "Don't call us, we'll call you."[2] The lower-level modules provide the implementation for interfaces that are declared within, and called by, the upper-level modules.

Using this inversion of ownership, `PolicyLayer` is unaffected by any changes to `MechanismLayer` or `UtilityLayer`. Moreover, `PolicyLayer` can be reused in any context that defines lower-level modules that conform to the `PolicyServiceInterface`. Thus, by inverting the dependencies, we have created a structure, which is simultaneously more flexible, durable, and mobile.

Depend On Abstractions

A somewhat more naive, yet still very powerful, interpretation of the DIP is the simple heuristic: "Depend on abstractions." Simply stated, this heuristic recommends that you should not depend on a concrete class—that all relationships in a program should terminate on an abstract class or an interface.

According to this heuristic,

- No variable should hold a pointer or reference to a concrete class.
- No class should derive from a concrete class.
- No method should override an implemented method of any of its base classes.

Certainly this heuristic is usually violated at least once in every program. Somebody has to create the instances of the concrete classes, and whatever module does that will depend on them.[3] Moreover, there seems no reason to follow this heuristic for classes that are concrete but nonvolatile. If a concrete class is not going to change very much, and no other similar derivatives are going to be created, then it does very little harm to depend on it.

2. [Sweet85].

3. Actually, there are ways around this if you can use strings to create classes. Java allows this. So do several other languages. In such languages, the names of the concrete classes can be passed into the program as configuration data.

For example, in most systems the class that describes a string is concrete. In Java, for example, it is the concrete class `String`. This class is not volatile. That is, it does not change very often. Therefore it does no harm to depend directly on it.

However, most concrete classes that *we* write as part of an application program are volatile. It is *those* concrete classes that we do not want to depend directly on. Their volatility can be isolated by keeping them behind an abstract interface.

This is not a complete solution. There are times when the interface of a volatile class must change, and this change must be propagated to the abstract interface that represents the class. Such changes break through the isolation of the abstract interface.

This is the reason that the heuristic is a bit naive. If, on the other hand, we take the longer view that the client classes declare the service interfaces that they need, then the only time the interface will change is when the *client* needs the change. Changes to the classes that implement the abstract interface will not affect the client.

A Simple Example

Dependency inversion can be applied wherever one class sends a message to another. For example, consider the case of the `Button` object and the `Lamp` object.

The `Button` object senses the external environment. On receiving the `Poll` message, it determines whether or not a user has "pressed" it. It doesn't matter what the sensing mechanism is. It could be a button icon on a GUI, a physical button being pressed by a human finger, or even a motion detector in a home security system. The `Button` object detects that a user has either activated or deactivated it.

The `Lamp` object affects the external environment. On receiving a `TurnOn` message, it illuminates a light of some kind. On receiving a `TurnOff` message, it extinguishes that light. The physical mechanism is unimportant. It could be an LED on a computer console, a mercury vapor lamp in a parking lot, or even the laser in a laser printer.

How can we design a system such that the `Button` object controls the `Lamp` object? Figure 11-3 shows a naive design. The `Button` object receives `Poll` messages, determines if the button has been pressed, and then simply sends the `TurnOn` or `TurnOff` message to the `Lamp`.

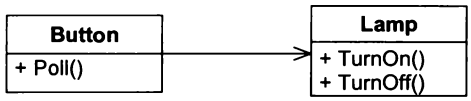


Figure 11-3 Naive Model of a Button and a Lamp

Why is this naive? Consider the Java code that is implied by this model (Listing 11-1). Note that the `Button` class depends directly on the `Lamp` class. This dependency implies that `Button` will be affected by changes to `Lamp`. Moreover, it will not be possible to reuse `Button` to control a `Motor` object. In this design, `Button` objects control `Lamp` objects, and *only* `Lamp` objects.

Listing 11-1

Button.java

```
public class Button
{
  private Lamp itsLamp;
  public void poll()
  {
    if (/*some condition*/)
      itsLamp.turnOn();
  }
}
```

This solution violates the DIP. The high-level policy of the application has not been separated from the low-level implementation. The abstractions have not been separated from the details. Without such a separation, the high-level policy automatically depends on the low-level modules, and the abstractions automatically depend on the details.

Finding the Underlying Abstraction

What is the high-level policy? It is the abstraction that underlies the application, the truths that do not vary when the details are changed. It is the system *inside* the system—it is the metaphor. In the `Button/Lamp` example, the underlying abstraction is to detect an on/off gesture from a user and relay that gesture to a target object. What mechanism is used to detect the user gesture? Irrelevant! What is the target object? Irrelevant! These are details that do not impact the abstraction.

The design in Figure 11-3 can be improved by inverting the dependency on the `Lamp` object. In Figure 11-4, we see that the `Button` now holds an association to something called a `ButtonServer`. `ButtonServer` provides abstract methods that `Button` can use to turn something on or off. `Lamp` implements the `ButtonServer` interface. Thus, `Lamp` is now doing the depending, rather than being depended on.

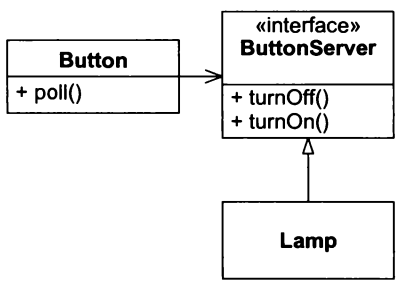


Figure 11-4 Dependency Inversion Applied to the Lamp

The design in Figure 11-4 allows a `Button` to control any device that is willing to implement the `ButtonServer` interface. This gives us a great deal of flexibility. It also means that `Button` objects will be able to control objects that have not yet been invented.

However, this solution also puts a constraint on any object that needs to be controlled by a `Button`. Such an object *must* implement the `ButtonServer` interface. This is unfortunate because these objects may also want to be controlled by a `Switch` object or some object other than a `Button`.

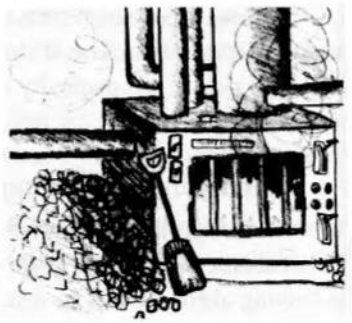
By inverting the direction of the dependency and making the `Lamp` do the depending instead of being depended on, we have made `Lamp` depend on a different detail—`Button`. Or have we?

`Lamp` certainly depends on `ButtonServer`, but `ButtonServer` does not depend on `Button`. Any kind of object that knows how to manipulate the `ButtonServer` interface will be able to control a `Lamp`. Thus, the dependency is in name only. And we can fix that by changing the name of `ButtonServer` to something a bit more generic like `SwitchableDevice`. We can also ensure that `Button` and `SwitchableDevice` are kept in separate libraries, so that the use of `SwitchableDevice` does not imply the use of `Button`.

In this case, nobody owns the interface. We have the interesting situation where the interface can be used by lots of different clients and implemented by lots of different servers. Thus, the interface needs to stand alone without belonging to either group. In C++, we would put it in a separate namespace and library. In Java we would put it in a separate package.[4]

4. In dynamic languages like Smalltalk, Pyrhon, or Ruby, the interface simply wouldn't exist as an explicit source-code entity.

The Furnace Example



Let's look at a more interesting example. Consider the software that might control the regulator of a furnace. The software can read the current temperature from an IO channel and instruct the furnace to turn on or off by sending commands to a different IO channel. The structure of the algorithm might look something like Listing 11-2.

Listing 11-2

Simple algorithm for a thermostat

```
#define TERMOMETER 0x86
#define FURNACE    0x87
#define ENGAGE     1
#define DISENGAGE  0

void Regulate(double minTemp, double maxTemp)
{
  for(;;)
  {
    while (in(THERMOMETER) > minTemp)
      wait(1);
    out(FURNACE,ENGAGE);

    while (in(THERMOMETER) < maxTemp)
      wait(1);
    out(FURNACE,DISENGAGE);
  }
}
```

The high-level intent of the algorithm is clear, but the code is cluttered with lots of low-level details. This code could never be reused with different control hardware.

This may not be much of a loss since the code is very small. But even so, it is a shame to have the algorithm lost for reuse. We'd rather invert the dependencies and see something like Figure 11-5.

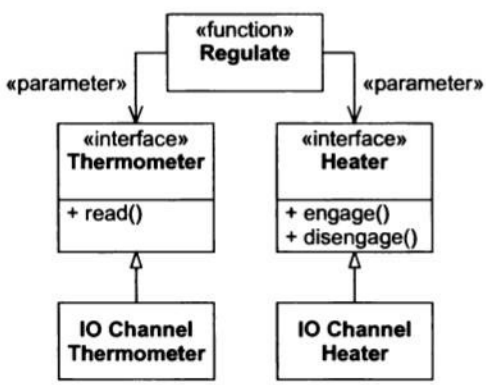


Figure 11-5 Generic Regulator

This shows that the regulate function takes two arguments that are both interfaces. The `Thermometer` interface can be read, and the `Heater` interface can be engaged and disengaged. This is all the `Regulate` algorithm needs. Now it can be written as shown in Listing 11-3.

Listing 11-3

Generic Regulator

```
void Regulate(Thermometer& t, Heater& h,
              double minTemp, double maxTemp)
{
  for(;;)
  {
    while (t.Read() > minTemp)
      wait(1);
    h.Engage();

    while (t.Read() < maxTemp)
      wait(1);
    h.Disengage();
  }
}
```

This has inverted the dependencies such that the high-level regulation policy does not depend on any of the specific details of the thermometer or the furnace. The algorithm is nicely reusable.

Dynamic v. Static Polymorphism

We have achieved the inversion of the dependencies, and made `Regulate` generic, through the use of dynamic polymorphism (i.e., abstract classes or interfaces). However, there is another way. We could have used the static form of polymorphism afforded by C++ templates. Consider Listing 11-4.

Listing 11-4

```
template <typename THERMOMETER, typename HEATER>
class Regulate(THERMOMETER& t, HEATER& h,
               double minTemp, double maxTemp)
{
  for(;;)
  {
    while (t.Read() > minTemp)
      wait(1);
    h.Engage();

    while (t.Read() < maxTemp)
      wait(1);
    h.Disengage();
  }
}
```

This achieves the same inversion of dependencies without the overhead (or flexibility) of dynamic polymorphism. In C++, the `Read`, `Engage`, and `Disengage` methods could all be nonvirtual. Moreover, any class that declares these methods can be used by the template. They do not have to inherit from a common base.

As a template, `Regulate` does not depend on any particular implementation of these functions. All that is required is that the class substituted for `HEATER` have an `Engage` and a `Disengage` method and that the class substituted for `THERMOMETER` have a `Read` function. Thus, those classes must implement the interface defined by the template. In other words, both `Regulate`, and the classes that `Regulate` uses, must agree on the same interface, and they both depend on that agreement.

Static polymorphism breaks the source-code dependency nicely, but it does not solve as many problems as does dynamic polymorphism. The disadvantages of the template approach are (1) The types of HEATER and THERMOMETER cannot be changed at runtime; and (2) The use of a new kind of HEATER or THERMOMETER will force recompilation and redeployment. So unless you have an extremely stringent requirement for speed, dynamic polymorphism should be preferred.

Conclusion

Traditional procedural programming creates a dependency structure in which policy depends on detail. This is unfortunate since the policies are then vulnerable to changes in the details. Object-oriented programming inverts that dependency structure such that both details and policies depend on abstraction, and service interfaces are often owned by their clients.

Indeed, it is this inversion of dependencies that is the hallmark of good object-oriented design. It doesn't matter what language a program is written in. If its dependencies are inverted, it has an OO design. If its dependencies are not inverted, it has a procedural design.

The principle of dependency inversion is the fundamental low-level mechanism behind many of the benefits claimed for object-oriented technology. Its proper application is necessary for the creation of reusable frameworks. It is also critically important for the construction of code that is resilient to change. Since the abstractions and details are all isolated from each other, the code is much easier to maintain.

Bibliography

1. Booch, Grady. *Object Solutions*. Menlo Park, CA: Addison–Wesley, 1996.
2. Gamma, et al. *Design Patterns*. Reading, MA: Addison–Wesley, 1995.
3. Sweet. Richard E. The Mesa Programming Environment. *SIGPLAN Notices*, 20(7) (July 1985): 216–229.

12

ISP: The Interface-Segregation Principle

This principle deals with the disadvantages of "fat" interfaces. Classes that have "fat" interfaces are classes whose interfaces are not cohesive. In other words, the interfaces of the class can be broken up into groups of methods. Each group serves a different set of clients. Thus, some clients use one group of member functions, and other clients use the other groups.

The ISP acknowledges that there are objects that require noncohesive interfaces; however, it suggests that clients should not know about them as a single class. Instead, clients should know about abstract base classes that have cohesive interfaces.

Interface Pollution

Consider a security system. In this system, there are `Door` objects that can be locked and unlocked, and which know whether they are open or closed. (See Listing 12-1.)

Listing 12-1

```
Security Door
class Door
{
  public:
    virtual void Lock()    = 0;
    virtual void Unlock() = 0;
    virtual bool IsDoorOpen() = 0;
};
```

This class is abstract so that clients can use objects that conform to the `Door` interface, without having to depend on particular implementations of `Door`.

Now consider that one such implementation, `TimedDoor`, needs to sound an alarm when the door has been left open for too long. In order to do this, the `TimedDoor` object communicates with another object called a `Timer`. (See Listing 12-2.)

Listing 12-2

```
class Timer
{
  public:
    void Register(int timeout, TimerClient* client);
};

class TimerClient
{
  public:
    virtual void TimeOut() = 0;
};
```

When an object wishes to be informed about a time-out, it calls the `Register` function of the `Timer`. The arguments of this function are the time of the time-out, and a pointer to a `TimerClient` object whose `TimeOut` function will be called when the time-out expires.

How can we get the `TimerClient` class to communicate with the `TimedDoor` class so that the code in the `TimedDoor` can be notified of the time-out? There are several alternatives. Figure 12-1 shows a naive solution. We force `Door`, and therefore `TimedDoor`, to inherit from `TimerClient`. This ensures that `TimerClient` can register itself with the `Timer` and receive the `TimeOut` message.

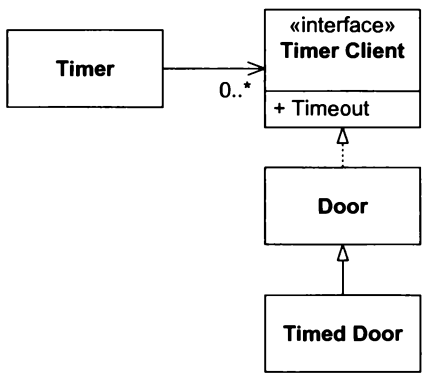


Figure 12-1 Timer Client at Top of Hierarchy

Although this solution is common, it is not without problems. Chief among these is that the `Door` class now depends on `TimerClient`. Not all varieties of `Door` need timing. Indeed, the original `Door` abstraction had nothing whatever to do with timing. If timing-free derivatives of `Door` are created, those derivatives will have to provide degenerate implementations for the `TimeOut` method—a potential violation of the LSP. Moreover, the applications that use those derivatives will have to import the definition of the `TimerClient` class, even though it is not used. That smells of Needless Complexity and Needless Redundancy.

This is an example of interface pollution, a syndrome that is common in statically typed languages like C++ and Java. The interface of `Door` has been polluted with a method that it does not require. It has been forced to incorporate this method solely for the benefit of one of its subclasses. If this practice is pursued, then every time a derivative needs a new method, that method will be added to the base class. This will further pollute the interface of the base class, making it "fat."

Moreover, each time a new method is added to the base class, that method must be implemented (or allowed to default) in derived classes. Indeed, an associated practice is to add these interfaces to the base class giving them degenerate implementations, specifically so that derived classes are not burdened with the need to implement them. As we learned previously, such a practice can violate the LSP, leading to maintenance and reusability problems.

Separate Clients Mean Separate Interfaces

`Door` and `TimerClient` represent interfaces that are used by completely different clients. `Timer` uses `TimerClient`, and classes that manipulate doors use `Door`. Since the clients are separate, the interfaces should remain separate, too. Why? Because clients exert forces on the interfaces they use.

The Backwards Force Applied by Clients On Interfaces

When we think of forces that cause changes in software, we normally think about how changes to interfaces will affect their users. For example, we would be concerned about the changes to all the users of `TimerClient` if the `TimerClient` interface changed. However, there is a force that operates in the other direction. Sometimes it is the *user* that forces a change to the interface.

For example, some users of `Timer` will register more than one time-out request. Consider the `TimedDoor`. When it detects that the `Door` has been opened, it sends the `Register` message to the `Timer`, requesting a time-out. However, before that time-out expires, the door closes, remains closed for a while, and then opens again. This causes us to register a *new* time-out request before the old one has expired. Finally, the first time-out request expires and the `TimeOut` function of the `TimedDoor` is invoked. The `Door` alarms falsely.

We can correct this situation by using the convention shown in Listing 12-3. We include a unique `timeOutId` code in each time-out registration, and we repeat that code in the `TimeOut` call to the `TimerClient`. This allows each derivative of `TimerClient` to know which time-out request is being responded to.

Listing 12-3

```
Timer with ID
class Timer
{
  public:
    void Register(int timeout,
                  int timeOutId,
                  TimerClient* client);
};

class TimerClient
{
  public:
    virtual void TimeOut(int timeOutId) = 0;
};
```

Clearly this change will affect all the users of `TimerClient`. We accept this since the lack of the `timeOutId` is an oversight that needs correction. However, the design in Figure 12-1 will also cause `Door`, and all clients of `Door` to be affected by this fix! This smells of Rigidity and Viscosity. Why should a bug in `TimerClient` have *any* affect on clients of `Door` derivatives that do not require timing? When a change in one part of the program affects other completely unrelated parts of the program, the cost and repercussions of changes become unpredictable, and the risk of fallout from the change increases dramatically.

ISP: The Interface-Segregation Principle

> *Clients should not be forced to depend on methods that they do not use.*

When clients are forced to depend on methods that they don't use, then those clients are subject to changes to those methods. This results in an inadvertent coupling between all the clients. Said another way, when a client depends on a class that contains methods that the client does not use, but that other clients *do* use, then that client will be

affected by the changes that those other clients force upon the class. We would like to avoid such couplings where possible, and so we want to separate the interfaces.

Class Interfaces v. Object Interfaces

Consider the `TimedDoor` again. Here is an object which has two separate interfaces used by two separate clients—`Timer` and the users of `Door`. These two interfaces *must* be implemented in the same object, since the implementation of both interfaces manipulates the same data. So how can we conform to the ISP? How can we separate the interfaces when they must remain together?

The answer to this lies in the fact that clients of an object do not need to access it through the interface of the object. Rather, they can access it through delegation or through a base class of the object.

Separation through Delegation

One solution is to create an object that derives from `TimerClient` and delegates to the `TimedDoor`. Figure 12-2 shows this solution.

When the `TimedDoor` wants to register a time-out request with the `Timer`, it creates a `DoorTimerAdapter` and registers it with the `Timer`. When the `Timer` sends the `TimeOut` message to the `DoorTimerAdapter`, the `DoorTimerAdapter` delegates the message back to the `TimedDoor`.

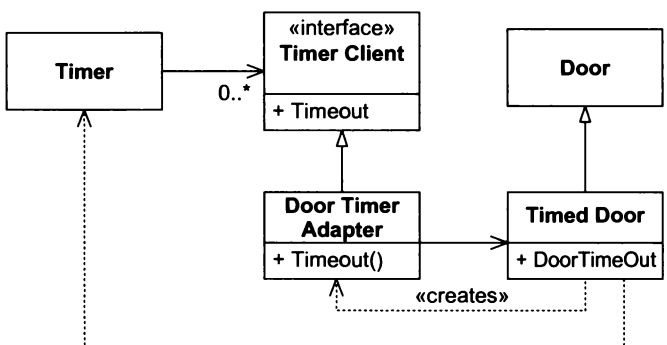


Figure 12-2 Door Timer Adapter

This solution conforms to the ISP and prevents the coupling of `Door` clients to `Timer`. Even if the change to `Timer` shown in Listing 12-3 were to be made, none of the users of `Door` would be affected. Moreover, `TimedDoor` does not have to have the exact same interface as `TimerClient`. The `DoorTimerAdapter` can *translate* the `TimerClient` interface into the `TimedDoor` interface. Thus, this is a very general purpose solution. (See Listing 12-4.)

Listing 12-4

`TimedDoor.cpp`

```
class TimedDoor : public Door
{
  public:
    virtual void DoorTimeOut(int timeOutId);
};

class DoorTimerAdapter : public TimerClient
{
  public:
    DoorTimerAdapter(TimedDoor& theDoor)
```

```
    : itsTimedDoor(theDoor)
    {}

    virtual void TimeOut(int timeOutId)
    {itsTimedDoor.DoorTimeOut(timeOutId);}

  private:
    TimedDoor& itsTimedDoor;
};
```

However, this solution is also somewhat inelegant. It involves the creation of a new object every time we wish to register a time-out. Moreover, the delegation requires a very small, but still nonzero, amount of runtime and memory. There are application domains, such as embedded real-time control systems, in which runtime and memory are scarce enough to make this a concern.

Separation through Multiple Inheritance

Figure 12-3 and Listing 12-5 show how multiple inheritance can be used to achieve the ISP. In this model, `TimedDoor` inherits from both `Door` and `TimerClient`. Although clients of both base classes can make use of `TimedDoor`, neither actually depends on the `TimedDoor` class. Thus, they use the same object through separate interfaces.

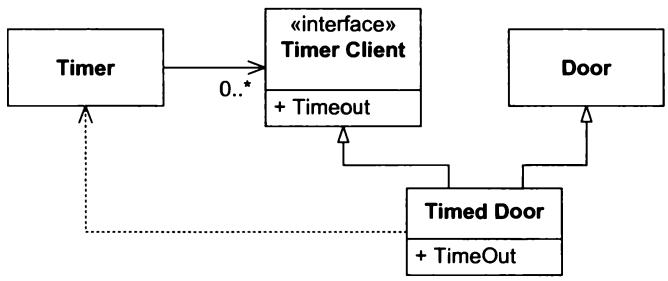


Figure 12-3 Multiply inherited Timed Door

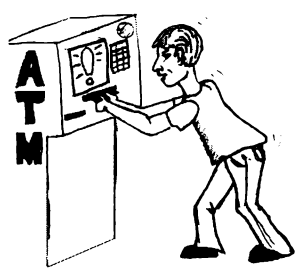
Listing 12-5

`TimedDoor.cpp`

```
class TimedDoor : public Door, public TimerClient
{
  public:
    virtual void TimeOut(int timeOutId);
};
```

This solution is my normal preference. The only time I would choose the solution in Figure12-2 over Figure 12-3 is if the translation performed by the `DoorTimerAdapter` object were necessary, or if different translations were needed at different times.

The ATM User Interface Example



Now let's consider a slightly more significant example. The traditional automated teller machine (ATM) problem. The user interface of an ATM machine needs to be very flexible. The output may need to be translated into many different languages. It may need to be presented on a screen, or on a braille tablet, or spoken out a speech synthesizer. Clearly this can be achieved by creating an abstract base class that has abstract methods for all the different messages that need to be presented by the interface.

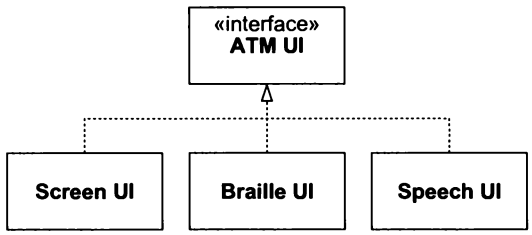


Figure 12-4

Consider also that each different transaction that the ATM can perform is encapsulated as a derivative of the class `Transaction`. Thus, we might have classes such as `DepositTransaction`, `WithdrawalTransaction`, and `TransferTransaction`. Each class invokes methods of the `UI`. For example, in order to ask the user to enter the amount he wishes to deposit, the `DepositTransaction` object invokes the `RequestDepositAmount` method of the `UI` class. Likewise, in order to ask the user how much money he wants to transfer between accounts, the `TransferTransaction` object calls the `RequestTransferAmount` method of `UI`. This corresponds to the diagram in Figure 12-5.

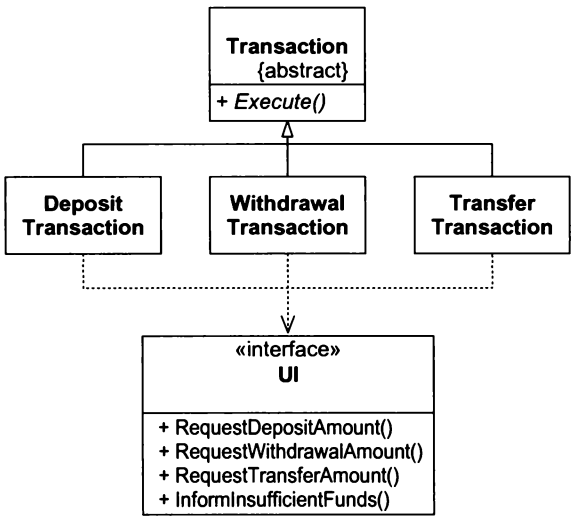


Figure 12-5 ATM Transaction Hierarchy

Notice that this is precisely the situation that the ISP tells us to avoid. Each transaction is using methods of the UI that no other class uses. This creates the possibility that changes to one of the derivatives of `Transaction` will force corresponding change to the UI, thereby affecting all the other derivatives of `Transaction` and every other class that depends on the `UI` interface. Something smells like Rigidity and Fragility around here.

For example, if we were to add a `PayGasBillTransaction`, we would have to add new methods to UI in order to deal with the unique messages that this transaction would want to display. Unfortunately, since `DepositTransaction`, `WithdrawalTransaction`, and `TransferTransaction` all depend on the `UI` interface, they must all be recompiled. Worse, if the transactions were all deployed as components in separate DLLs or shared libraries, then those components would have to be redeployed, even though none of their logic was changed. Can you smell the Viscosity?

This unfortunate coupling can be avoided by segregating the `UI` interface into individual interfaces such as `DepositUI`, `WithdrawUI`, and `TransferUI`. These separate interfaces can then be multiply inherited into the final `UI` interface. Figure 12-6 and Listing 12-6 show this model.

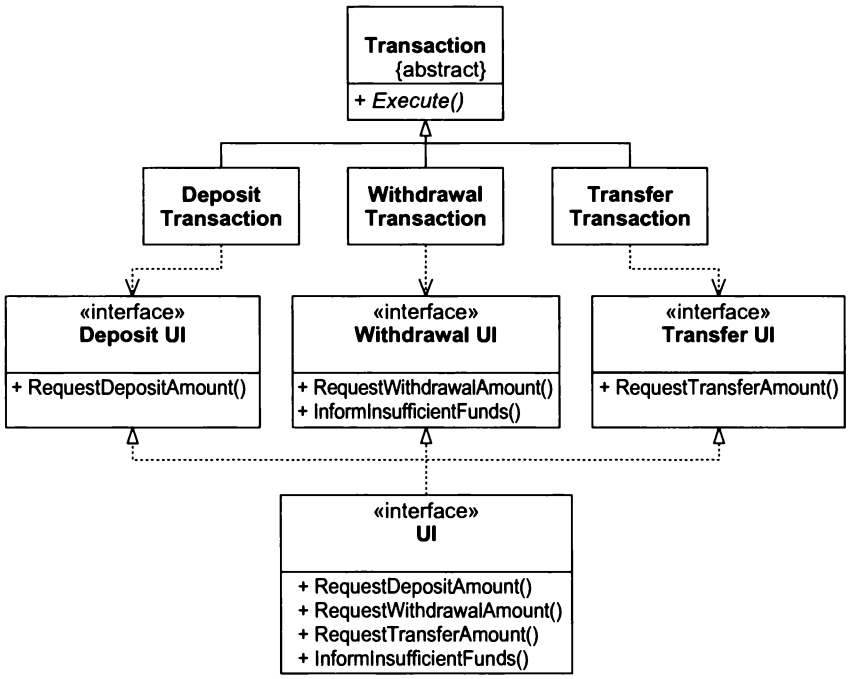


Figure 12-6 Segregated ATM UI Interface

Whenever a new derivative of the `Transaction` class is created, a corresponding base class for the abstract `UI` interface will be needed, and so the `UI` interface and all its derivatives must change. However, these classes are not widely used. Indeed, they are probably only used by `main` or whatever process boots the system and creates the concrete `UI` instance. So the impact of adding new `UI` base classes is minimized.

Listing 12-6

Segregated ATM UI Interface

```
class DepositUI
{
  public:
    virtual void RequestDepositAmount() = 0;
};

class DepositTransaction : public Transaction
{
  public:
    DepositTransaction(DepositUI& ui)
    : itsDepositUI(ui)
    {}

    virtual void Execute()
    {
      ...
      itsDepositUI.RequestDepositAmount();
```

```
      ...
    }
  private:
    DepositUI& itsDepositUI;
};

class WithdrawalUI
{
  public:
    virtual void RequestWithdrawalAmount() = 0;
};

class WithdrawalTransaction : public Transaction
{
  public:
    WithdrawalTransaction(WithdrawalUI& ui)
    : itsWithdrawalUI(ui)
    {}

    virtual void Execute()
    {
      ...
      itsWithdrawalUI.RequestWithdrawalAmount();
      ...
    }
  private:
    WithdrawalUI& itsWithdrawalUI;
};

class TransferUI
{
  public:
    virtual void RequestTransferAmount() = 0;
};

class TransferTransaction : public Transaction
{
  public:
    TransferTransaction(TransferUI& ui)
    : itsTransferUI(ui)
    {}

    virtual void Execute()
    {
      ...
      itsTransferUI.RequestTransferAmount();
      ...
    }
  private:
    TransferUI& itsTransferUI;
};

class UI : public DepositUI
         , public WithdrawalUI
         , public TransferUI
```

```
{
  public:
    virtual void RequestDepositAmount();
    virtual void RequestWithdrawalAmount();
    virtual void RequestTransferAmount();
};
```

A careful examination of Listing 12-6 will show one of the issues with ISP conformance that was not obvious from the `TimedDoor` example. Note that each transaction must somehow know about its particular version of the `UI`. `DepositTransaction` must know about `DepositUI`, `WithdrawTransaction` must know about `WithdrawUI`, etc. In Listing 12-6, I have addressed this issue by forcing each transaction to be constructed with a reference to its particular UI. Note that this allows me to employ the idiom in Listing 12-7.

Listing 12-7

Interface Initialization Idiom

```
UI Gui; // global object;

void f()
{
    DepositTransaction dt(Gui);
}
```

This is handy, but it also forces each transaction to contain a reference member to its UI. Another way to address this issue is to create a set of global constants as shown in Listing 12-8. Global variables are not always a symptom of a poor design. In this case they provide the distinct advantage of easy access. Since they are references, it is impossible to change them in any way. Therefore they cannot be manipulated in a way that would surprise other users.

Listing 12-8

Seperate Global Pointers

```
// in some module that gets linked in
// to the rest of the app.

static UI Lui; // non-global object;
DepositUI&   GdepositUI   = Lui;
WithdrawalUI& GwithdrawalUI = Lui;
TransferUI&  GtransferUI  = Lui;

// In the depositTransaction.h module

class WithdrawalTransaction : public Transaction
{
  public:

    virtual void Execute()
    {
      ...
      GwithdrawalUI.RequestWithdrawalAmount();
      ...
    }
};
```

In C++, one might be tempted to put all the globals in Listing 12-8 into a single class in order to prevent pollution of the global namespace. Listing 12-9 shows such an approach. This, however, has an unfortunate effect. In order to use `UIGlobals`, you must `#include ui_globals.h`. This, in turn, `#includes depositUI.h`, `withdrawUI.h`, and `transferUI.h`. This means that any module wishing to use any of the UI interfaces transitively depends on all of them— exactly the situation that the ISP warns us to avoid. If a change is made to any of the UI interfaces, all modules that `#include "ui_globals.h"` are forced to recompile. The `UIGlobals` class has recombined the interfaces that we had worked so hard to segregate!

Listing 12-9

Wrapping the Globals in a class

```
// in ui_globals.h

#include "depositUI.h"
#include "withdrawalUI.h"
#include "transferUI.h"

class UIGlobals
{
  public:
    static WithdrawalUI& withdrawal;
    static DepositUI&   deposit;
    static TransferUI&  transfer
};

// in ui_globals.cc

static UI Lui; // non-global object;
DepositUI&   UIGlobals::deposit   = Lui;
WithdrawalUI& UIGlobals::withdrawal = Lui;
TransferUI&  UIGlobals::transfer  = Lui;
```

The Polyad v. the Monad

Consider a function `g` that needs access to both the `DepositUI` and the `TransferUI`. Consider also that we wish to pass the UIs into this function. Should we write the function prototype like this?

```
void g(DepositUI&, TransferUI&);
```

Or should we write it like this?

```
void g(UI&);
```

The temptation to write the latter (monadic) form is strong. After all, we know that in the former (polyadic) form, both arguments will refer to the *same object*. Moreover, if we were to use the polyadic form, its invocation might look like this:

```
g(ui, ui);
```

Somehow, this seems perverse.

Perverse or not, the polyadic form is often preferable to the monadic form. The monadic form forces `g` to depend on every interface included in `UI`. Thus, when `WithdrawUI` changes, `g` and all clients of `g` could be affected. This is more perverse than `g(ui,ui)`! Moreover, we cannot be sure that both arguments of `g` will *always*

refer to the same object! In the future, it may be that the interface objects are separated for some reason. The fact that all interfaces are combined into a single object is information that g does not need to know. Thus, I prefer the polyadic form for such functions.

Grouping Clients. Clients can often be grouped together by the service methods they call. Such groupings allow segregated interfaces to be created for each group instead of each client. This greatly reduces the number of interfaces that the service has to implement, and it also prevents the service from depending on each client type.

Sometimes, the methods invoked by different groups of clients will overlap. If the overlap is small, then the interfaces for the groups should remain separate. The common functions should be declared in all the overlapping interfaces. The server class will inherit the common functions from each of those interfaces, but it will implement them only once.

Changing Interfaces. When object-oriented applications are maintained, the interfaces to existing classes and components often change. There are times when these changes have a huge impact and force the recompilation and redeployment of a very large part of the system. This impact can be mitigated by adding new interfaces to existing objects, rather than changing the existing interface. Clients of the old interface that wish to access methods of the new interface can query the object for that interface, as shown in Listing 12-10.

Listing 12-10

```
void Client(Service* s)
{
  if (NewService* ns = dynamic_cast<NewService*>(s))
  {
    // use the new service interface
  }
}
```

As with all principles, care must be taken not to overdo it. The spectre of a class with hundreds of different interfaces, some segregated by client and others segregated by version, would be frightening indeed.

Conclusion

Fat classes cause bizarre and harmful couplings between their clients. When one client forces a change on the fat class, all the other clients are affected. Thus, clients should only have to depend on methods that they actually call. This can be achieved by breaking the interface of the fat class into many client-specific interfaces. Each client-specific interface declares only those functions that its particular client, or client group, invoke. The fat class can then inherit all the client-specific interfaces and implement them. This breaks the dependence of the clients on methods that they don't invoke, and it allows the clients to be independent of each other.

Bibliography

1. Gamma, et al. *Design Patterns*. Reading, MA: Addison–Wesley, 1995.

SECTION 3

The Payroll Case Study



The time has come for our first major case study. We have studied practices and principles. We have discussed the essence of design. We have talked about testing and planning. Now we need to do some real work.

In the next several chapters, we are going to explore the design and implementation of a batch payroll system. A rudimentary specification of that system is included later. As part of that design and implementation, we will find ourselves making use of several different design patterns. Among those patterns are COMMAND, TEMPLATE METHOD, STRATEGY, SINGLETON, NULL OBJECT, FACTORY, and FACADE. These patterns are the topic of the next several chapters. Then, in Chapter 18, we work through the design and implementation of the Payroll problem.

There are several ways to read through this case study:

- Read straight through, first learning the design patterns and then seeing how they are applied to the payroll problem.
- If you know the patterns and are not interested in a review, then go right to Chapter 18.
- Read Chapter 18 first, and then go back and read through the chapters that describe the patterns that were used.
- Read Chapter 18 in bits. When it talks about a pattern with which you are unfamiliar, read through the chapter that describes that pattern, and then return to Chapter 18.
- Indeed, there are no rules. Pick, or invent, the strategy that works best for you.

Rudimentary Specification of the Payroll System

The following are some of the notes we took while conversing with our customer.

This system consists of a database of the employees in the company and their associated data, such as time cards. The system must pay each employee. Employees must be paid the correct amount, on time, by the method that they specify. Also, various deductions must be taken from their pay.

- Some employees work by the hour. They are paid an hourly rate that is one of the fields in their employee record. They submit daily time cards that record the date and the number of hours worked. If they work more than 8 hours per day, they are paid 1.5 times their normal rate for those extra hours. They are paid every Friday.
- Some employees are paid a flat salary. They are paid on the last working day of the month. Their monthly salary is one of the fields in their employee record.
- Some of the salaried employees are also paid a commission based on their sales. They submit sales receipts that record the date and the amount of the sale. Their commission rate is a field in their employee record. They are paid every other Friday.
- Employees can select their method of payment. They may have their paychecks mailed to the postal address of their choice; they may have their paychecks held for pickup by the Paymaster; or they can request that their paychecks be directly deposited into the bank account of their choice.
- Some employees belong to the union. Their employee record has a field for the weekly dues rate. Their dues must be deducted from their pay. Also, the union may assess service charges against individual union members from time to time. These service charges are submitted by the union on a weekly basis and must be deducted from the appropriate employee's next pay amount.
- The payroll application will run once each working day and pay the appropriate employees on that day. The system will be told to what date the employees are to be paid, so it will calculate payments from the last time the employee was paid up to the specified date.

Exercise

Before you continue, you might find it instructive to design the payroll system on your own, now. You might want to sketch some initial UML diagrams. Better yet, you might want to implement the first few use cases test-first. Apply the principles and practices we've learned so far, and try to create a balanced and healthy design.

If you are going to do this, then take a look at the use cases that follow. Otherwise skip them, they'll be presented again in the Payroll chapter.

Use Case 1: Add New Employee

A new employee is added by the receipt of an `AddEmp` transaction. This transaction contains the employee's name, address, and assigned employee number. The transaction has the following three forms:

```
AddEmp <EmpID> "<name>" "<address>" H <hourly-rate>
AddEmp <EmpID> "<name>" "<address>" S <monthly-salary>
AddEmp <EmpID> "<name>" "<address>" C <monthly-salary> <commission-rate>
```

The employee record is created with its fields assigned appropriately.

Alternative:
An error in the transaction structure

If the transaction structure is inappropriate, it is printed out in an error message, and no action is taken.

Use Case 2: Deleting an Employee

Employees are deleted when a `DelEmp` transaction is received. The form of this transaction is as follows:

```
DelEmp <EmpID>
```

When this transaction is received, the appropriate employee record is deleted.

Alternative:
Invalid or unknown `EmpID`

If the `<EmpID>` field is not structured correctly, or if it does not refer to a valid employee record, then the transaction is printed with an error message, and no other action is taken.

Use Case 3: Post a `Time Card`

Upon receipt of a `TimeCard` transaction, the system will create a time-card record and associate it with the appropriate employee record.

```
TimeCard <Empld> <date> <hours>
```

Alternative 1:
The selected employee is not hourly

The system will print an appropriate error message and take no further action.

Alternative 2:
An error in the transaction structure

The system will print an appropriate error message and take no further action.

Use Case 4: Posting a `Sales Receipt`

Upon receipt of the `SalesReceipt` transaction, the system will create a new sales-receipt record and associate it with the appropriate commissioned employee.

```
SalesReceipt <EmpID> <date> <amount>
```

Alternative 1:
The selected employee is not commissioned

The system will print an appropriate error message and take no further action.

Alternative 2:
An error in the transaction structure

The system will print an appropriate error message and take no further action.

Use Case 5: Posting a Union Service Charge

Upon receipt of this transaction, the system will create a service-charge record and associate it with the appropriate union member.

```
ServiceCharge <memberID> <amount>
```

Alternative:
Poorly formed transaction

If the transaction is not well formed or if the `<memberID>` does not refer to an existing union member, then the transaction is printed with an appropriate error message.

Use Case 6: Changing Employee Details

Upon receipt of this transaction, the system will alter one of the details of the appropriate employee record. There are several possible variations to this transaction.

`ChgEmp <EmpID> Name <name>`	Change Employee Name
`ChgEmp <EmpID> Address <address>`	Change Employee Address
`ChgEmp <EmpID> Hourly <hourlyRate>`	Change to Hourly
`ChgEmp <EmpID> Salaried <salary>`	Change to Salaried
`ChgEmp <EmpID> Commissioned <salary> <rate>`	Change to Commissioned
`ChgEmp <EmpID> Hold`	Hold Paycheck
`ChgEmp <EmpID> Direct <bank> <account>`	Direct Deposit
`ChgEmp <EmpID> Mail <address>`	Mail Paycheck
`ChgEmp <EmpID> Member <memberID> Dues <rate>`	Put Employee in Union
`ChgEmp <EmpID> NoMember`	Remove Employee from Union

Alternative:
Transaction Errors

If the structure of the transaction is improper or `<EmpID>` does not refer to a real employee or `<memberID>` already refers to a member, then print a suitable error and take no further action.

Use Case 7: Run the Payroll for Today

Upon receipt of the Payday transaction, the system finds all those employees that should be paid upon the specified date. The system then determines how much they are owed and pays them according to their selected payment method.

```
Payday <date>
```

13

COMMAND and ACTIVE OBJECT



No man has received from nature the right to command his fellow human beings.

—Denis Diderot (1713–1784)

Of all the design patterns that have been described over the years, COMMAND impresses me as one of the simplest and most elegant. As we shall see, the simplicity is deceptive. The range of uses that COMMAND may be put to is probably without bound.

The simplicity of COMMAND, as shown in Figure 13-1, is almost laughable. Listing 13-1 doesn't do much to dampen the levity. It seems absurd that we can have a pattern that consists of nothing more than an interface with one method.

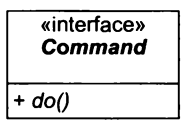


Figure 13-1 COMMAND Pattern

Listing 13-1

Command.java

```
public interface Command
{
  public void do();
}
```

But, in fact, a very interesting line has been crossed by this pattern. And it is in the crossing of this line that all the interesting complexity lies. Most classes associate a suite of methods with a corresponding set of variables. The COMMAND pattern does not do this. Rather, it encapsulates a function free of any variables.

In strict object-oriented terms, this is anathema—it smacks of functional decomposition. It elevates the role of a function to the level of a class. Blasphemy! Yet, at this boundary where two paradigms clash, interesting things start to occur.

Simple Commands

Several years ago, I consulted for a large firm that made photocopiers. I was helping one of their development teams with the design and implementation of the embedded real-time software that drove the inner workings of a new copier. We stumbled on the idea of using the COMMAND pattern to control the hardware devices. We created a hierarchy that looked something like Figure 13-2.

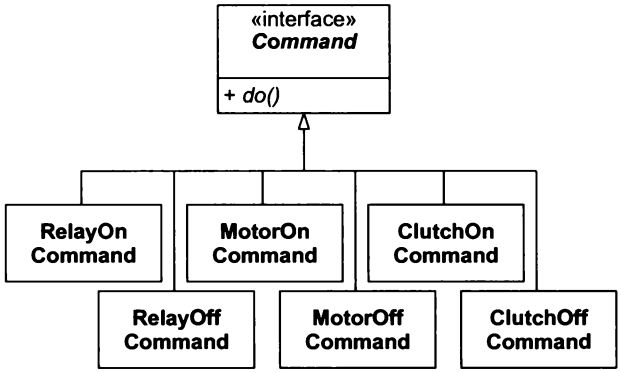


Figure 13-2 Some Simple Commands for the Copier software

The role of these classes should be obvious. When you call `do()` on a `RelayOnCommand`, it turns some relay on. When you call `do()` on a `MotorOffCommand`, it turns some motor off. The address of the motor or relay is passed into the object as an argument to its constructor.

With this structure in place, we could now pass `Command` objects around the system and `do()` them without knowing precisely what kind of `Command` they represented. This led to some interesting simplifications.

The system was event driven. Relays opened or closed, motors started or stopped, and clutches engaged or disengaged based on certain events that took place in the system. Many of those events were detected by sensors. For example, when an optical sensor determined that a sheet of paper had reached a certain point in the paper path, we'd need to engage a certain clutch. We were able to implement this by simply binding the appropriate `ClutchOnCommand` to the object that controlled that particular optical sensor. (See Figure 13-3.)

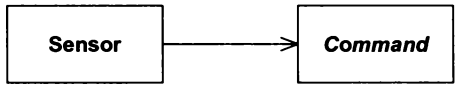


Figure 13-3 A Command driven by a Sensor

This simple structure has an enormous advantage. The `Sensor` has no idea what it is doing. Whenever it detects an event, it simply calls `do()` on the `Command` that it is bound to. This means that the `Sensors` don't have to know about individual clutches or relays. They don't have to know the mechanical structure of the paper path. Their function becomes remarkably simple.

The complexity of determining which relays to close when certain sensors declare events has moved to an initialization function. At some point during the initialization of the system, each `Sensor` is bound to an appropri-

ate `Command`. This puts all the *wiring*[1] in one place and gets it out of the main body of the system. Indeed, it would be possible to create a simple text file that described which `Sensors` were bound to which `Commands`. The initialization program could read this file and build the system appropriately. Thus, the *wiring* of the system could be determined completely outside the program, and it could be adjusted without recompilation.

By encapsulating the *notion* of a command, this pattern allowed us to decouple the logical interconnections of the system from the devices that were being connected. This was a huge benefit.

Transactions

Another common use of the COMMAND pattern, and one that we will find useful in the Payroll problem, is in the creation and execution of transactions. Imagine, for example, that we are writing the software that maintains a database of employees. (See Figure 13-4.) There are a number of operations that users can apply to that database. They can add new employees, delete old employees, or change the attributes of existing employees.

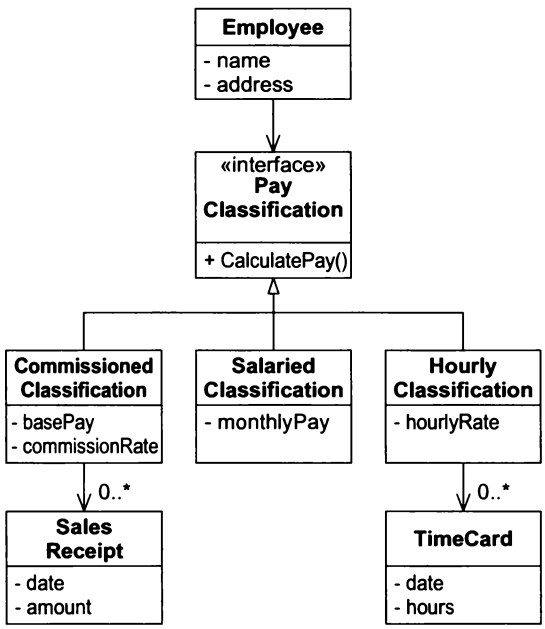


Figure 13-4 Employee Database

When a user decides to add a new employee, that user must specify all the information needed to successfully create the employee record. Before acting on that information, the system needs to verify that the information is syntactically and semantically correct. The COMMAND pattern can help with this job. The `command` object acts as a repository for the unvalidated data, implements the validation methods, and implements the methods that finally execute the transaction.

For example, consider Figure 13-5. The `AddEmployeeTransaction` contains the same data fields that `Employee` contains. It also holds a pointer to a `PayClassification` object. These fields and object are created from the data that the user specifies when directing the system to add a new employee.

The `validate` method looks over all the data and makes sure it makes sense. It checks it for syntactic and semantic correctness. It may even check to ensure that the data in the transaction are consistent with the existing state of the database. For example, it might ensure that no such employee already exists.

1. The logical interconnections between the `Sensors` and `Commands`.

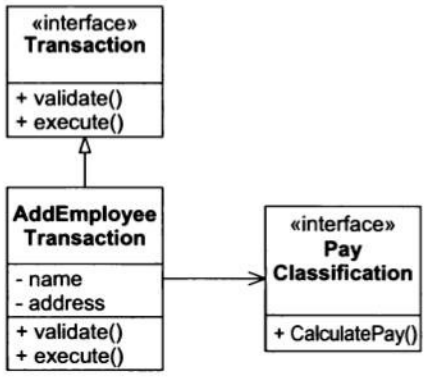


Figure 13-5 `AddEmployee` Transaction

The `execute` method uses the validated data to update the database. In our simple example, a new `Employee` object would be created and loaded with the fields from the `AddEmployeeTransaction` object. The `PayClassification` object would be moved or copied into the `Employee`.

Physical and Temporal Decoupling

The benefit this gives us is in the dramatic decoupling between the code that procures the data from the user, the code that validates and operates on that data, and the business objects themselves. For example, one might expect the data for adding a new employee to be procured from a dialog box in some GUI. It would be a shame if the GUI code contained the validation and execution algorithms for the transaction. Such a coupling would prevent that validation and execution code from being used with other interfaces. By separating the validation and execution code into the `AddEmployeeTransaction` class, we have physically decoupled that code from the procurement interface. What's more, we've separated the code that knows how to manipulate the logistics of the database from the business entities themselves.

Temporal Decoupling

We have also decoupled the validation and execution code in a different way. Once the data have been procured, there is no reason why the validation and execution methods must be called immediately. The transaction objects can be held in a list and validated and executed much later.

Suppose we have a database that must remain unchanged during the day. Changes may only be applied during the hours between midnight and 1 A.M. It would be a shame to have to wait until midnight and then have to rush to type in all the commands before 1 A.M. It would be much more convenient to type in all the commands, have them validated on the spot, and then executed later at midnight. The COMMAND pattern gives us this ability.

UNDO

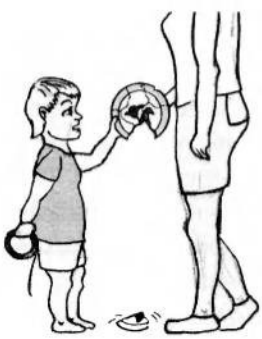


Figure 13-6 adds the `undo()` method to the COMMAND pattern. It stands to reason that if the `do()` method of a `Command` derivative can be implemented to remember the details of the operation it performs, then the `undo()` method can be implemented to undo that operation and return the system to its original state.

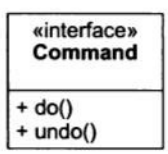


Figure 13-6 Undo variation of the COMMAND Pattern.

Imagine, for example, an application that allows the user to draw geometric shapes on the screen. A toolbar has buttons that allow the user to draw circles, squares, rectangles, etc. Let's say that the user clicks on the *draw circle* button. The system creates a `DrawCircleCommand` and then calls `do()` on that command. The `DrawCircleCommand` object tracks the user's mouse waiting for a click in the drawing window. On receiving that click, it sets the click point as the center of the circle and proceeds to draw an animated circle at that center with a radius that tracks the current mouse position. When the user clicks again, the `DrawCircleCommand` stops animating the circle and adds the appropriate circle object to the list of shapes currently displayed on the canvas. It also stores the ID of the new circle in some private variable of its own. Then it returns from the `do()` method. The system then pushes the expended `DrawCirlceCommand` on the stack of completed commands.

Later, the user clicks the undo button on the toolbar. The system pops the completed commands stack and calls `undo()` on the resulting `Command` object. on receiving the `undo()` message, the `DrawCircleCommand` object deletes the circle matching the saved ID from the list of objects currently displayed on the canvas.

With this technique, you can easily implement the `undo` command in nearly any application. The code that knows how to undo a command is always right next to the code that knows how to perform the command.

ACTIVE OBJECT

One of my favorite uses of the COMMAND pattern is the ACTIVE OBJECT pattern.[2] This is a very old technique for implementing multiple threads of control. It has been used, in one form or another, to provide a simple multitasking nucleus for thousands of industrial systems.

The idea is very simple. Consider Listings 13-2 and 13-3. An `ActiveObjectEngine` object maintains a linked list of `Command` objects. Users can add new commands to the engine, or they can call `run()`. The `run()` function simply walks through the linked list executing and removing each command.

Listing 13-2

`ActiveObjectEngine.java`

```
import java.util.LinkedList;
import java.util.Iterator;

public class ActiveObjectEngine
{
  LinkedList itsCommands = new LinkedList();

  public void addCommand(Command c)
  {
    itsCommands.add(c);
  }

  public void run()
  {
    while (!itsCommands.isEmpty())
    {
      Command c = (Command) itsCommands.getFirst();
      itsCommands.removeFirst();
      c.execute();
    }
  }
}
```

2. [Lavender96].